

LQCD_Master 双代理编排库分析报告

格点 QCD 自然语言到 GPU 计算的自动化 agent 编排体系

opencode analy

2026 年 8 月 17 日

摘要

本报告对 refer/git-rep/LQCD_Master (LQCD Master, 下称“目标库”) 做只读全库分析: **Planner (物理方案规划)** 与 **Executor (代码生成)** 双代理将自然语言物理任务翻译为可执行的 PyQUDA 程序与 Slurm 提交脚本, 并配 5 个 LQCD 领域技能与 70 项物理基准任务。全库 5600 个文件、约 185.6 万行 (精确求和 1,855,830 行), 按目录汇总计数 + 抽查比例饱和覆盖。三视角结论: **主体结构**为”入口层 → 编排核心层 → 提示词/技能/工具/配置层 → 基准/实验层”八层组织; **各部分关系**为”run.py 装配编排器 → planner/executor 消费 prompts+skills+configs → submit_tool 提交 Slurm”的依赖链; **项目思路**是”人机协同 + 自动纠错 + 技能路由”三支柱的可复现科学计算管线。主题分析以”如何让 agent 自动算一个 2pt 关联函数”为实例, 按 **A 代码工作框架 15 步**重现双代理编排全流程。

目录

1 仓库概览	2
1.1 目录结构	3
1.2 规模统计与 git 信息	3
2 主体结构	3
2.1 入口层与编排核心层	4
2.2 提示词层与技能层	4
2.3 工具层、配置层、基准层与实验层	4
3 各部分关系	5
3.1 引用链: 入口到编排核心	5
3.2 编排核心到知识/工具层	5
3.3 双代理内部的提示词调用链	5
3.4 技能路由的配置配对关系	5
3.5 配置与提交脚本的生成链	6
3.6 基准与实验的验证闭环	6
3.7 演进关系	6

	2
4 项目思路	6
4.1 设计意图	6
4.2 演进脉络	7
4.3 典型 workflow	7
5 主题分析: LQCD_Master 双代理编排全流程	7
5.1 A1 目的与前期准备	7
5.2 A2 输入是什么	7
5.3 A3 输出应该是什么	8
5.4 A4 整体框架	8
5.5 A5 关键细节	8
5.6 A6 任务如何正确执行	9
5.7 A7 实际执行情况	9
5.8 A8 实际输出情况	9
5.9 A9 结果分析	9
5.10 A10 综合分析	9
5.11 A11 结尾总结	10
5.12 A12 结尾评价	10
5.13 A13 补充说明	10
5.14 A14 参考源	10
5.15 A15 附录与附件	10
6 关键代码/文档片段	10
6.1 入口装配	10
6.2 Planner 三段式核心	11
6.3 技能注入 (SkillMessageAssembler)	12
6.4 Executor 生成提示词 (骨架约束)	13
6.5 基准任务原文	13
6.6 Slurm 配置	13
6.7 技能核心链 (Wick 收缩)	14
6.8 静态分析入口	14
7 参考源清单	14
8 结论与局限	16

1 仓库概览

目标库位于 `refer/git-rep/LQCD_Master/`, 英文 README 自称”Agentic Scientific Computing for Lattice QCD —Natural Language to GPU Computation” (`README.md:1-10`)。它不是独立 git 仓

库（无自带 .git），被 PyQCD 主仓库跟踪，故 git 历史反映主仓库而非本库自身——报告中 git 信息一律注明“非独立仓库”。

1.1 目录结构

```

1 LQCD_Master/
2 |-- run.py           # CLI 入口 (133 行)
3 |-- README.md        # 项目说明
4 |-- requirements.txt  # openai/prompt_toolkit/pyyaml/numpy/opt_einsum
5 |-- .env.template    # LLM API / Serper key 模板
6 |-- core_architecture/ # 编排核心: orchestrator(816) planner(379) executor(1219)
7 |-- prompts/         # 提示词: planner/(solve/critique/rewrite) executor/(generate/critique/rewrite)
8 |-- skills/          # 5 个 LQCD 技能 SKILL.md + reference/ 工作示例
9 |-- utils/           # llm_client / skill_utils / static_check / generate_einsum / ...
10 |-- configs/         # config.yaml / ensemble_presets.yaml / skills.yaml
11 |-- benchmark/       # 70 任务 + standard 手写 PyQUDA 参考实现
12 |-- experiments/     # 多模型实验记录 (GPT-5.4/DeepSeek/Claude_Code/多强子等)
13 |-- docs/           # 既有 pure 报告 (纯 QCD Wick 代码生成, 1 份 tex+pdf)

```

1.2 规模统计与 git 信息

指标	实测值
文件总数	5600
总行数（全部文本精确求和）	1,855,830
markdown 文档数	197
Python 文件数	356
Shell 脚本数	450
yaml 配置数	159
txt（任务/数据/日志）	4432
独立 git 仓库	否（被 PyQCD 主仓库跟踪）

表 1: 目标库实测统计（数据来源: find/wc 实测; git 信息: git -C 上溯至主仓库）

按目录汇总（文件数 / 行数）：benchmark 153/15144、configs 3/212、core_architecture 4/2415、experiments 5335/1572939、prompts 15/527、skills 12/3348、utils 71/260521、docs 2/386。其中 experiments 占文件与行数绝对多数（含 New_multi_hadron 下 100,829 行级的生成型 sink_*_2pt.py，属于自动生成的 Wick 收缩大文件，仅索引）。参考知识库声明：本库内 docs/（2 文件，既有 pure_qcd_wick_codegen_20260815 产物）已引入并仅索引；本库无 books/refer/references 等目录，未引入。

2 主体结构

本节回答“仓库是什么、如何分层”。按“入口 → 编排 → 知识/工具 → 基准/实验”八层组织，每层职责一句话。

层次	目录	职责
入口层	<code>run.py</code>	CLI 入口：解析参数、装配编排器并启动全流程
编排核心层	<code>core_architecture/</code>	PlannerAgent + ExecutorAgent + WorkflowOrchestrator 三体编排
提示词层	<code>prompts/</code>	按阶段 (solve/generate/critique/rewrite) 的 system/user prompt 模板
技能层	<code>skills/</code>	5 个 LQCD 领域技能 SKILL.md + reference 工作示例
工具层	<code>utils/</code>	LLM 客户端、技能注册/路由、静态分析、Wick einsum 代码生成器
配置层	<code>configs/</code> + <code>.env</code>	Slurm/ensemble/技能路由/LLM API 配置
基准层	<code>benchmark/</code>	70 项验证任务 + 手写标准参考实现与数值数据
实验层	<code>experiments/</code>	多 LLM 骨干实验记录与逐任务产物

表 2: 主体结构分层 (数据来源: 全库索引实测; 行数见 README.md:65-84 项目结构节)

2.1 入口层与编排核心层

入口层仅 `run.py` 一个文件 (133 行), 职责极简: 解析 CLI、装配四个组件 (LLM 客户端、内置工具、Slurm 提交工具、编排器) 后调用 `orchestrator.run()` (`run.py:90-117`)。编排核心层三个类的职责划分清晰: `WorkflowOrchestrator` (816 行) 负责阶段总控与 Slurm 队列监视; `PlannerAgent` (379 行) 负责物理方案; `ExecutorAgent` (1219 行) 负责代码生成、静态分析与提交脚本渲染。行数之比 816:379:1219 直观反映“方案最薄、代码生成最厚”的职责分布。

2.2 提示词层与技能层

提示词层共 15 个文件、527 行, 按 `planner/` (solve/critique/rewrite 三子阶段, 各带 system+user) 与 `executor/` (generate/critique/rewrite) 双代理对称组织, 另含 `plan_template.yaml` 与每阶段 `skill_routing.md`。技能层 5 个 SKILL.md 共 1926 行: `lqcd-analysis`(740) > `pyquda-tool`(614) > `pyquda-gauge`(222) > `lqcd-physics-correlator`(199) > `lqcd-physics-spectrum`(151)。

2.3 工具层、配置层、基准层与实验层

工具层 `utils/` (71 文件) 是代码量最大工程层, 其中 `generate_einsum/` (Wick 收缩 einsum 代码生成器) 含 100,829 行级自动生成文件, 是行数主要来源 (`utils/generate_einsum/two_pt/sink_pnn_full.py:1`)。配置层 3 个 yaml 共 212 行: `config.yaml` (Slurm/runtime)、`ensemble_presets.yaml` (7 套 ensemble)、`skills.yaml` (技能路由)。基准层 70 任务分 5 类 (README.md:108-118); 实验层 5335 文件按“骨干模型 × 观测量类 × 任务号”组织。

本节小结: 八层结构以“编排核心”为轴, 提示词/技能/工具/配置四层供其消费, 基准与实验层作为验证闭环。下一节回答这些层之间如何连接。

3 各部分关系

本节回答”各部分怎么连”。依赖主链：

要点

依赖主链： `run.py` → `core_architecture/orchestrator` → (`planner.py` + `executor.py`) → `prompts/` + `skills/` + `configs/` + `utils/` → `utils/submit_tool` → Slurm 集群 → `experiments/` 产物

3.1 引用链：入口到编排核心

`run.py:90-93` 在 `main()` 中以 `from ... import ...` 延迟导入编排器与四个工具类 (`WorkflowOrchestrator/LQCDLLMClient/ BuiltinToolClient/SlurmSubmitTool`)。编排器构造时再装配 `PlannerAgent` 与 `ExecutorAgent` (`core_architecture/orchestrator.py:50-60`)，形成 `run.py` → `orchestrator` → `planner/executor` 引用链。

3.2 编排核心到知识/工具层

3.3 双代理内部的提示词调用链

项目思路

Planner 内链： `planner.py` 的 `run()` 依序调 `_run_stage("solve")` → `_run_stage("critique")` → `_run_stage("rewrite")`，每阶段 `_load_system_prompt` 读 `prompts/planner/` 对应模板 (`core_architecture/planner.py:72-112`、`core_architecture/planner.py:269-275`)。**Executor 内链：** `executor.py` 的 `generate_bundle` 依序走 `_run_executor_stage` → `_run_critique_stage` → `_run_static_analysis_stage` → `_run_rewriter_stage` (`core_architecture/executor.py:102-159`、`core_architecture/executor.py:428-524`)。双代理均经 `utils/skill_utils.SkillRunner` 注入技能、经 `utils/llm_client` 调用大模型。

3.4 技能路由的配置配对关系

`configs/skills.yaml:1-10` 声明 `enabled_skills`、`stage_forced_skills` (`planner` 强制 `lqcd-physics-correlator+` `pyquda-gauge`，`executor` 强制 `pyquda-tool+pyquda-gauge`) 与 `skill_tools` (技能 → 工具映射)。该配置被 `SkillRouter` (`utils/skill_utils.py:133-190`) 消费：`candidate_skills` 依 `enabled/forced/disabled` 三集合求候选，`tools_for_skills` 把技能映射为 OpenAI 工具。同一技能名同时出现在 `skills` 目录、`skills.yaml` 与 `prompts/*/skill_routing.md`，三处构成”技能目录声明 ↔ 路由配置 ↔ 阶段路由指引”的配对三角 (确证)。

3.5 配置与提交脚本的生成链

`executor._render_submit_script` (`core_architecture/executor.py:1113-1160`) 从 `configs/config.yaml` 的 `script/slurm/runtime/ensemble` 四节渲染 `sbatch` 脚本: `#SBATCH` 头、`module load`、`exports`、`mpirun` 启动行。`utils/config_utils` 的 `build_prompt_submit_config_yaml` 把配置拼进 `prompt` (`utils/config_utils.py:126-149`)。形成 `config.yaml` → `submit` 脚本渲染 → `Slurm` 提交 生成链。

3.6 基准与实验的验证闭环

`benchmark/*/task/*.txt` 是自然语言任务, `benchmark/*/standard/benchmark.py` 是手写 PyQUDA 参考实现 (如 `benchmark/2pt_local/standard/benchmark.py:1-30` 定义 C24P29 ensemble 参数), `benchmark_data/*.txt` 是参考数值。实验层 `experiments/*/summary.md` 逐任务对比标准值与 agent 产物, 形成 `task` 文本 → agent 管线产物 → 与 `standard benchmark` 数据比对 → 准确率统计 验证闭环 (`experiments/LQCD_Master_GPT_5.4/summary.md:8-30`)。

3.7 演进关系

由于非独立 git 仓库, 无法从 git 历史直接观察演进; 从文件结构可推演演进痕迹 (推断): `utils/generate_einsum/three_pt/_to_delete/` (旧版 codegen 归档)、`codegen_mesonold.py`、`skills/pyquda-tool/SKILL_old_0605.md` (旧技能备份)、`experiments/` 下 `test1..test4` 与 `executor_v1/v2` 版本目录, 表明“新版本并行保留 + 旧版归档”的演进管理惯例。

本节小结: 依赖关系呈“入口薄、核心厚、消费端多层”的扇形; 技能路由与配置配对是 数据驱动 的关键机制。下一节回答为什么这样设计。

4 项目思路

本节回答“为什么长这样”。三支柱设计意图 + 演进脉络 + 典型工作流。

4.1 设计意图

支柱一: 人机协同 (human in the loop)。README 明示“Every stage has a checkpoint —the system never submits to the cluster without confirmation” (`README.md:57-61`)。编排器在 `planner` 与 `executor` 两阶段后均有 `_checkpoint_planner/_checkpoint_executor` 人类确认点 (`core_architecture/orchestrator.py:109-133`、`core_architecture/orchestrator.py:160-199`)。

支柱二: 自动纠错 (auto-debugging)。静态分析发现 errors 时 `executor` 自动重写, 上限 `executor_static_check_rounds`; warnings 不阻断 (`README.md:59-61`、`configs/config.yaml:1-2`)。静态检查由 AST 驱动的 `utils/static_check.py` 实现 (`utils/static_check.py:10-25`)。

支柱三: 技能路由 (skill routing)。领域知识按阶段相关性注入 LLM prompt (`README.md:61`)。物理推导 (`lqcd-physics-*`) 进 `planner`, 代码/工具 (`pyquda-*`) 进 `executor`, 分工写入 `configs/skills.yaml:4-8`。

4.2 演进脉络

推断 (依据文件结构, 非 git 历史): 从 experiments/ 中 Claude_Code_GPT_5.4/test1..test4 (早期探索)、LQCD_Master_GPT_5.4 (主验证)、LQCD_Master_DeepSeek (多骨干对照) 到 New_DA_diagonal、New_multi_hadron (新物理任务扩展, 含 $p+n+\Lambda$ 多强子 9 夸克收缩) 可见”探索 → 全量验证 → 多骨干 → 新任务”的演进线。技能层 SKILL_old_0605.md 与 generate_einsum 的 _to_delete/ 表明技能与代码生成器持续重构。

4.3 典型 workflow

一次任务的工作流 (以 2pt 关联函数为例, 详见第 5 节): 自然语言任务 → run.py 装配 → **Planner** solve/critique/rewrite 产出 plan.yaml+summary.md → 人工 checkpoint → **Executor** generate (经 SkillRunner 注入技能、调 generate_einsum 工具) + critique + 静态分析 + rewrite 产出 main.py + 提交脚本 → 人工 checkpoint → test 提交 + queue/scontrol 监视 → full 提交 → 数据 txt 落盘。

项目思路

一句话思路: 这个仓库把”格点 QCD 专家的物理推理 + 代码工程 + 集群运维”编码进 **双代理 prompt 体系 + 技能库 + 工具链**, 用人工 checkpoint 兜底, 实现”自然语言 → GPU 计算”的可复现、可对照基准的自动化科研管线。

5 主题分析: LQCD_Master 双代理编排全流程

主题为一项具体工作——”如何让 agent 自动算一个 2pt 关联函数”。判定为 **A 代码/软件工作** (对象以编排代码、prompt 模板、脚本为主), 按 15 步框架完整组织; 涉及物理推导的内容并入对应步骤。

5.1 A1 目的与前期准备

为什么存在: LQCD 计算脚本的编写长期依赖专家手写, 门槛高、易错。该库将”自然语言物理任务 → 可执行 PyQUDA 程序 + Slurm 脚本”的翻译工作自动化 (README.md:12)。**前置条件:** Python 3.10+、PyQUDA+QUDA、Slurm GPU 分区、OpenAI 兼容 LLM 端点 (README.md:160-167), API key 经 .env 提供 (.env.template:1-8)。**环境准备:** pip install -r requirements.txt, cp .env.template .env (README.md:16-28)。[确证]

5.2 A2 输入是什么

三层输入: **任务文本**——命令行 --task 或交互输入 (run.py:19-24), 也可直接吃基准任务文件 (benchmark/2pt_local/task/task_1.txt:1-5: ”Calculate the two-point function of a pion with the operator $\bar{u}\gamma_5 d$... point source at [0,0,0,0] ... stout-smeared (1, 0.125, 4) ...”); **配置**——configs/config.yaml (Slurm 分区/资源)、ensemble_presets.yaml (C24P29 ensemble: latt_size [24,24,24,72]、夸克质量、clover 系数, benchmark/2pt_local/standard/benchmark.py:17-25) 、

skills.yaml; **环境事实**——executor 会探测 cfg 文件存在性与 gauge 头 (core_architecture/executor.py:633-700)。[确证]

5.3 A3 输出应该是什么

Planner 输出: plan.yaml (标准字段 task/ensemble/physics/measurement/output, 模板 prompts/planner/plan_template.yaml:1-65) 与 summary.md (物理目标/策略/合理性说明), 落盘于 run_dir/planner/ (core_architecture/planner.py:146-149)。**Executor 输出:** main.py (单文件顺序脚本)、submit_test.sh、submit_full.sh、notes, 打包为 bundle (core_architecture/executor.py:150-158)。**最终产物:** 提交集群后逐 cfg 的 *.txt 数据文件与 trajectory_full.json 轨迹 (run.py:123)。[确证]

5.4 A4 整体框架

四层主流程 (依赖链见第 3 节): `run.py` → `orchestrator.run()` → `_run_planner_stage` → `_run_executor_stage` → `_run_test_submission_stage`。编排器 `run()` 仅 23 行, 三阶段串行 (core_architecture/orchestrator.py:74-96): Planner 产出方案 → Executor 产出代码与提交脚本 → (非 test 模式) test 提交 + 监视。文件组织上”编排核心薄、提示词/技能/工具厚”, 主流程走向如下:

```

1  def run(self) -> dict[str, Any]:
2      print("===== [ Planner ] =====")
3      planner_output, planner_version = self._run_planner_stage()
4      print()
5      print("===== [ Executor ] =====")
6      bundle, exec_version = self._run_executor_stage(planner_output)
7      if self.test_mode:
8          self.state["test_submission"] = {
9              "approved": False,
10             "result": {"skipped": True, "reason": "test_mode"},
11         }
12         self.state["full_submission"] = {
13             "approved": False,
14             "result": {"skipped": True, "reason": "not_submitted_in_main_workflow"},
15         }
16         self._save_state()
17     else:
18         self._run_test_submission_stage(bundle, exec_version, planner_output)
19
20     self.state["finished_at"] = datetime.now().isoformat()
21     self.state["status"] = "completed"
22     self._save_state()
23     return self.state

```

5.5 A5 关键细节

细节一: Planner 三段式 (solve→critique→rewrite)。先解、再自我批判、再改写, 且批判只标”会导致代码失败或物理错误”的问题 (prompts/planner/critique_system.md:1-9)。**细节二: plan**

覆盖校验。 REQUIRED_PLAN_KEYS 校验 plan_yaml 是否含 task/ensemble/physics/measurement/output/extras (core_architecture/planner.py:22-30、core_architecture/planner.py:151-153)。**细节三: executor 静态分析三路。** python 语法/AST + PyQUDA API 语义 + shell 脚本 (utils/static_check.py:10-25)。**细节四: generate_einsum 工具强制。** 所有 2pt/3pt 收缩必须调工具生成, 带 # FROM generate_einsum 水印供 critique 检查 (prompts/executor/generate_system.md:63-82)。**细节五: 缺失字段自动反馈重试。** bundle 验证失败时自动构造反馈走 rewriter (上限 2 次, core_architecture/executor.py:165-197)。**易错点: 禁止 main 函数/参数解析/类/异常处理, 脚本须单文件顺序展开** (prompts/executor/generate_system.md:34-46)。**[确证]**

5.6 A6 任务如何正确执行

标准做法: `pip install -r requirements.txt; cp .env.template .env` 填 key; 然后 `python run.py --task "Calculate the pion two-point correlator" --test --non-interactive` (README.md:16-28)。`--test` 只生成不提交, `--non-interactive` 跳过人工 checkpoint。基准复跑用 `experiments/run_scripts/batch_local2pt.sh` 等批处理脚本 (`experiments/run_scripts/batch_local2pt.sh:1`)。**[确证]**

5.7 A7 实际执行情况

未验证: 真实端到端运行需 LLM API key 与 Slurm 集群, 本分析为只读性质、未实际调用。只读侧实测: 对库内 69 个核心/工具/配置 Python 文件做 `py_compile` 语法检查, **OK=69, FAIL=0** (命令与结果见会话日志); 目录/文件/行数统计 (第 1.2 节) 与 README 声明 (70 任务、5 类 observable) 逐项核对一致 (README.md:108-118)。**[确证 (语法检查) / 未验证 (端到端)]**

5.8 A8 实际输出情况

引用仓库内真实实验产物目录 (完整路径见第 7 节参考源清单): 内含 `main.py` 与三个提交脚本 (`submit_test.sh`、`submit_full.sh`、`submit_50.sh`) 以及 401 个逐 cfg 的 pion 2pt 数据文件 (命名 `cfgNNNNN.txt`, `cfg 10000-20050`, 步长 50); `planner/` 下有 `plan.yaml` 与 `summary.md`。**[确证]**

5.9 A9 结果分析

结合实验汇总文件 (`experiments/LQCD_Master_GPT_5.4/summary.md:8-30`): 70 任务 63 个 Exact ($|\Delta| < 10^{-12}$ 或 $\text{rel} < 10^{-3}$)、3 个 sign flip (γ_5 厄米相位约定, 物理不受影响)、4 个失败, 总准确率 90.0%。2pt local 20/20 全对; 3pt baryon 最低 (10/15)。DeepSeek 骨干 80.0%, 失败集中在“代码未产生有效结果” (`experiments/LQCD_Master_DeepSeek/summary.md:8-30`)。解读: agent 对 **纯规范测量 (wilson loop)** 最稳 (100%), 对重子 **3pt 顺序源** 最不稳——与收缩拓扑复杂度成正比; 数值符号约定是独立风险源。**[确证]**

5.10 A10 综合分析

代码-对象映射: 自然语言任务 $\leftrightarrow$ 物理观测量; `plan.yaml` $\leftrightarrow$ 物理方案 (算符/传播子/源-汇); `main.py` $\leftrightarrow$ 单组态数据生产脚本; `submit_*.sh` $\leftrightarrow$ 集群作业; `generate_einsum` 工具 $\leftrightarrow$ Wick 收缩自

动生成；`static_check` ↔ 语法/API 语义审查。**深层原因**：双代理把”物理正确性”与”代码正确性”分离验证，避免一个模型同时承担两难任务；批判-改写循环模拟同行评审，比单次生成显著降低错误（对比实验 90% vs 80% 体现骨干差异，`README.md:126-134`）。[确证/推断]

5.11 A11 结尾总结

结论

双代理编排全流程是”规划 (Planner solve/critique/rewrite) → 生成 (Executor generate/critique/static-check/rewrite) → 提交监视 (test→full)”的闭环，辅以技能路由、工具强制 (`generate_einsum`) 与人工 checkpoint，将 70 项基准中的标准任务 (2pt/wilson/3pt meson) 做到 100% 级别准确率，重子 3pt 与符号约定是主要短板。

5.12 A12 结尾评价

优点：物理与代码双层评审、可复现基准闭环、多骨干可对比、产物完整落盘 (`main.py`/提交脚本/逐 `cfg` 数据/轨迹)。**可改进处**：批判标准偏保守 (只标致命问题，`minor` 统计注意项可能漏掉)；生成型大文件 (10 万行级 `sink`) 体积失控；符号翻转类错误仅靠后验比对发现，缺预防机制。[确证/推断]

5.13 A13 补充说明

假设：base 配置 (`partition/ensemble/QUDA` 路径) 须逐集群定制 (`README.md:150-157`)。**局限**：真实提交/运行未验证 (只读分析)；非独立 git 仓库故演进脉络为推断；`experiments` 与生成大文件仅抽查比例覆盖 (5335 文件，抽查 `task_1` 全链 + `summary` 汇总，比例约 3%)。[未验证/如实标注]

5.14 A14 参考源

本步证据汇总见第 7 节参考源清单；关键片段直引见第 6 节与 A4/A5 内嵌 `lstinputlisting`。

5.15 A15 附录与附件

`benchmark/2pt_local/task/task_1.txt` (任务原文，见第 6 节片段 5) 与 `configs/config.yaml` (Slurm 配置，见第 6 节片段 6)；完整产物清单见第 7 节参考源清单 (真实产物目录条目)。

6 关键代码/文档片段

全部片段用 `\lstinputlisting` 直引原文件，每段 ≤ 40 行。

6.1 人口装配

```

1  from core_architecture.orchestrator import WorkflowOrchestrator
2  from utils.llm_client import LQCDLLMClient
3  from utils.submit_tool import SlurmSubmitTool
4  from utils.tool_client import BuiltinToolClient
5
6  if args.task.endswith(".txt"):
7      with open(args.task, "r") as f:
8          task = f.read().strip()
9          args.task = task
10
11 task = prompt_task(args.task)
12
13 run_dir = resolve_run_dir(args.run_dir)
14 run_dir.mkdir(parents=True, exist_ok=True)
15
16 llm = LQCDLLMClient(dotenv_path=args.dotenv_path)
17 tool_client = BuiltinToolClient(dotenv_path=args.dotenv_path)
18 submit_tool = SlurmSubmitTool()
19 orchestrator = WorkflowOrchestrator(
20     task=task,
21     run_dir=run_dir,
22     llm_client=llm,
23     tool_client=tool_client,
24     submit_tool=submit_tool,
25     test_mode=args.test,
26     non_interactive=args.non_interactive,
27 )

```

6.2 Planner 三段式核心

```

1  def run(self, task: str) -> dict[str, Any]:
2      self._begin_iteration(version_label="v1", kind="initial", feedback=None)
3      self.solve_payload = self._run_stage(
4          stage="solve",
5          build_messages=lambda: self._build_stage_user_message(
6              stage="planner_solve",
7              payload={"task": task, "fixed_facts_yaml": self.fixed_facts_yaml},
8              task=task,
9          ),
10         task=task,
11     )
12
13 self.critique_payload = self._run_stage(
14     stage="critique",
15     build_messages=lambda: self._build_stage_user_message(
16         stage="planner_critique",
17         payload={"task": task, "solve_payload": self.solve_payload},
18         task=task,
19     ),
20     task=task,
21 )
22 self.critique_payload = self._normalize_critique_payload(self.critique_payload)
23

```

```

24     self.rewrite_payload = self._run_stage(
25         stage="rewrite",
26         build_messages=lambda: self._build_stage_user_message(
27             stage="planner_rewrite",
28             payload={
29                 "task": task,
30                 "solve_payload": self.solve_payload,
31                 "critique_payload": self.critique_payload,
32                 "fixed_facts_yaml": self.fixed_facts_yaml,
33             },
34             task=task,
35         ),
36         task=task,
37     )
38
39     output = self._to_planner_output(self.rewrite_payload, fallback=self.solve_payload)
40     self._persist_final(output)

```

6.3 技能注入 (SkillMessageAssembler)

```

1  class SkillMessageAssembler:
2      def __init__(self, max_inline_reference_chars: int = 24000):
3          self.max_inline_reference_chars = max_inline_reference_chars
4
5      def build_system_message(self, base_system_prompt: str, knowledge_skills: list[LoadedSkill]) -> str:
6          blocks = [base_system_prompt.strip()]
7          for skill in knowledge_skills:
8              blocks.append(self._skill_block("Knowledge skill", skill))
9          return "\n\n".join(block for block in blocks if block).strip()
10
11     def build_messages(
12         self,
13         *,
14         base_system_prompt: str,
15         knowledge_skills: list[LoadedSkill],
16         user_message: str,
17     ) -> list[dict[str, str]]:
18         system = self.build_system_message(base_system_prompt, knowledge_skills)
19         return [
20             {"role": "system", "content": system},
21             {"role": "user", "content": user_message.strip()},
22         ]
23
24     def _skill_block(self, title: str, skill: LoadedSkill) -> str:
25         sections = [
26             f"[{title}] {skill.name}",
27             f"Description: {skill.description}",
28             skill.body.strip(),
29         ]
30         references_text = self._inline_references(skill.references_dir)
31         if references_text:
32             sections.append(references_text)
33         return "\n\n".join(part for part in sections if part).strip()

```

6.4 Executor 生成提示词 (骨架约束)

```

1
2 Code structure and writing order:
3
4 - the script should be a single-file sequential execution, without a `main` function;
5 - do not define any helper functions;
6 - write in the following order (this is the universal skeleton for ALL LQCD code):
7     1. parameter definitions (hard-coded)
8     2. read gauge configuration
9     3. construct the Dirac operator (skip if gauge-only)
10    4. compute forward propagators (skip if gauge-only)
11    5. extract observable / compute contraction (task-specific)
12    6. save the result

```

6.5 基准任务原文

```

1 Calculate the two-point function of a pion with the operator \bar u gamma_5 d.
2 Use the point source at position [0,0,0,0].
3 Invert all propagators using stout-smeared gauge links with parameters (1, 0.125, 4).
4 Save the final result without any header or extra text to a txt file in the run directory.

```

6.6 Slurm 配置

```

1 workflow:
2     executor_static_check_rounds: 1
3     executor_dynamic_test_rounds: 1
4
5 script:
6     shell: /bin/bash
7
8 slurm:
9     partition: kshdexclu16
10    nodes: 1
11    gres: dcu:4
12    exclude: f17r3n00
13    ntasks_per_node: 4
14    ntasks_per_socket: 1
15    exclusive: true
16    array: "10000-10850:50"
17
18 ensemble:
19     preset: C24P29
20     cfg_num: ["10000"]
21
22 runtime:
23     module_purge: true
24     modules:
25         - apps/git/2.30.2
26         - compiler/cmake/3.23.3
27         - compiler/gnu/9.3.0

```

```

28 - compiler/intel/2017.5.239
29 - mpi/hpcx/2.11.0/intel-2017.5.239
30 - compiler/dtk/25.04.4

```

6.7 技能核心链 (Wick 收缩)

```

1  Given a physics observable such as hadron mass and decay constant that can be determined by local two-point
   ↪ correlation function (2pt), quasi-distribution amplitudes from nonlocal two-point correlation function, form
   ↪ factor that can be determined by three-point correlation function (3pt), ... ,
2  derive the complete chain:
3
4  **Observable  $\rightarrow$  Operator(s)  $\rightarrow$  Correlator(s)  $\rightarrow$  Wick contraction  $\rightarrow$ 
   ↪ Propagator(s)  $\rightarrow$  Einsum(s)**
5
6  so that downstream tools (PyQUDA) know exactly what to compute.
7
8  For three-point correlation function, this skill additionally provides the full sequential-source implementation chain
   ↪ from
9  operator definition to executable contraction code, including the two-dagger
10 convention, propagator requirements, and final contraction forms.
11
12 This skill stops once the correlator expression, propagator list, and
13 contraction/einsum structure are fixed.
14
15 **IMPORTANT**: The contraction/einsum structure is for REASONING only. Never include it in the planner's output — the
   ↪ executor will regenerate it via the generate_einsum tool. For Euclidean-time dependence,
16 backward-state structure, and fit templates, hand off to
17 `lqcd-physics-spectrum`.

```

6.8 静态分析入口

```

1  def analyze_bundle_static(
2      *,
3      main_program: str,
4      test_submit_script: str,
5      full_submit_script: str,
6  ) -> dict[str, Any]:
7      program_report = analyze_python_static(main_program)
8      pyquda_report = analyze_pyquda_usage(main_program)
9      test_report = analyze_shell_static(test_submit_script, script_name="submit_test.sh")
10     full_report = analyze_shell_static(full_submit_script, script_name="submit_full.sh")
11
12     return {
13         "main_errors": list(program_report["errors"]) + list(pyquda_report["errors"]),
14         "test_sh_errors": list(test_report["errors"]),
15         "full_sh_errors": list(full_report["errors"]),
16     }

```

7 参考源清单

参考源	类型	内容/作用
README.md:1-10	仓库	项目定位：自然语言到 GPU 计算
README.md:57-61	仓库	三支柱设计决策 (human-in-loop/auto-debug/skill routing)
README.md:108-118	仓库	70 任务基准分类
README.md:126-134	仓库	实验对比：GPT-5.4 90.0% vs DeepSeek 80.0%
run.py:17-25	仓库	CLI 参数定义
run.py:90-117	仓库	main() 组件装配与编排启动
core_architecture/orchestrator.py:17-28	仓库	终态 Slurm 状态集合
core_architecture/orchestrator.py:74-96	仓库	run() 三阶段串行
core_architecture/orchestrator.py:98-133	仓库	planner checkpoint/rewrite 循环
core_architecture/orchestrator.py:160-199	仓库	executor checkpoint 循环
core_architecture/orchestrator.py:201-281	仓库	test 提交与动态重写
core_architecture/orchestrator.py:283-343	仓库	squeue/scontrol 队列监视
core_architecture/planner.py:22-30	仓库	REQUIRED_PLAN_KEYS
core_architecture/planner.py:72-112	仓库	PlannerAgent.run solve/critique/rewrite
core_architecture/planner.py:146-153	仓库	产物持久化与覆盖校验
core_architecture/executor.py:102-159	仓库	generate_bundle 主流程
core_architecture/executor.py:165-197	仓库	缺字段自动反馈重试
core_architecture/executor.py:428-487	仓库	executor/critique/rewriter 子阶段
core_architecture/executor.py:633-700	仓库	输入事实采集 (cfg 探测)
core_architecture/executor.py:1113-1160	仓库	提交脚本渲染
utils/skill_utils.py:133-190	仓库	SkillRouter 路由逻辑
utils/skill_utils.py:215-247	仓库	SkillMessageAssembler 技能注入
utils/skill_utils.py:319-465	仓库	ToolRegistry 工具实现
utils/static_check.py:10-25	仓库	三路静态分析入口
utils/static_check.py:186-265	仓库	PyQUDA 调用 AST 分析
utils/submit_tool.py:10-53	仓库	sbatch 提交与 job_id 解析
prompts/planner/solve_system.md:1-15	仓库	物理学家 persona 与 standard/freeform 双模式
prompts/planner/critique_system.md:1-9	仓库	批判标准 (只标致命问题)

参考源	类型	内容/作用
prompts/executor/generate_system.md:34-46	仓库	单文件顺序脚本约束
prompts/executor/generate_system.md:63-82	仓库	generate_einsum 强制与水印
skills/lqcd-physics-correlator/SKILL.md:25-41	仓库	可观测 → 传播子 →einsum 链
skills/pyquda-tool/SKILL.md:1-30	仓库	PyQUDA 代码生成技能
configs/config.yaml:1-41	仓库	Slurm/runtime/ensemble 配置
configs/skills.yaml:1-10	仓库	技能路由配置
benchmark/2pt_local/task/task_1.txt:1-5	仓库	2pt pion 任务原文
benchmark/2pt_local/standard/benchmark.py:1-30	仓库	标准参考实现（C24P29 参数）
experiments/LQCD_Master_GPT_5.4/summary.md:8-30	仓库	70 任务 90.0% 结果
experiments/LQCD_Master_DeepSeek/summary.md:8-30	仓库	70 任务 80.0% 结果
experiments/LQCD_Master_GPT_5.4/2pt_local/task_1/executor_v1/	仓库	真实产物（main.py+401 cfg txt）
experiments/run_scripts/batch_local2pt.sh:1	仓库	基准批处理脚本
docs/pure_qcd_wick_codegen_20260815.tex:1	参考	既有 pure 报告（本库文档层）

8 结论与局限

结论

结构：八层组织——入口 (run.py)/编排核心 (core_architecture)/提示词 (prompts)/ 技能 (skills)/工具 (utils)/配置 (configs)/基准 (benchmark)/实验 (experiments)。**关系：**run.py→ 编排器 → 双代理 →prompts+skills+configs+utils→submit_tool→ 集群 → experiments 的扇形依赖链，技能路由为数据驱动枢轴。**思路：**人机协同 + 自动纠错 + 技能路由三支柱，把专家物理推理与代码工程编码为可复现管线；双代理把物理正确性与代码正确性分离，批判-改写模拟同行评审。**主题结论：**agent 自动算 2pt 关联函数的全流程（A1–A15）已完整走通，70 任务中标准任务近满分，重子 3pt 与符号约定为短板。

参考背景

本库 docs/ 既有 pure 报告 (pure_qcd_wick_codegen_20260815) 聚焦 Wick 收缩代码生成器, 与本文第 5 节 A5 中 generate_einsum 工具强制一节互为旁证: 该工具是 executor 正确生成 2pt/3pt 收缩代码的关键依赖。

局限与风险

未验证: 真实端到端运行 (LLM 调用 + Slurm 提交) 未执行, 仅做 69 个核心 py 文件语法检查 (OK=69/FAIL=0)。**推断:** 演进脉络基于文件结构推演 (非独立 git 仓库, 无本库自身历史)。**抽查比例:** experiments/ (5335 文件) 按目录汇总 + task_1 全链抽查, 未逐文件精读; 100,829 行级生成型 sink*_2pt.py 仅索引。参考知识库仅引入本库 docs/ (2 文件); 本库无 books/refer/references 等外部参考目录。